Agentic fMRIPrep

One-page repo-based summary of the application in /Users/manyashetty/Desktop/Agentic_fMRIPrep.

What It Is

A Python application that wraps fMRIPrep in a LangGraph-driven agent loop so command generation, execution, failure diagnosis, recovery, and post-run QA can happen with less manual intervention. The repo positions it as an autonomous preprocessing pipeline for BIDS-formatted neuroimaging data.

Who It's For

Primary user: a neuroimaging researcher or data engineer who runs fMRIPrep on BIDS datasets and wants help with configuration, retries, and quality checks.

What It Does

- Loads runtime settings from YAML, environment variables, and CLI overrides via a central Config object.
- Builds an fMRIPrep Docker command from repo config and dataset facts such as participant ID and fieldmap presence.
- Optionally uses local PDF manuals plus embeddings/Chroma for RAG-backed command generation when dependencies are available.
- Executes the generated command and captures stdout/stderr for downstream handling.
- Diagnoses failures with either an LLM or regex heuristics for common fMRIPrep, Docker, metadata, and memory issues.
- Applies command-level recovery fixes and loops through retries up to the configured maximum.
- Runs a post-execution QA step that compares expected anatomical input/output files and reports simple voxel-based checks when possible.

How It Works

Entry point: main.py parses CLI args, builds Config, validates paths, and invokes a compiled LangGraph app. **Core services:** agents/config_agent.py generates the Docker command; agents/orchestrator.py runs the planner-executor-detective-engineer loop; agents/diagnostic_agent.py interprets failures; agents/recovery_agent.py rewrites the command; config_loader.py resolves config; local docs live in data/docs/; vectors persist in database/vector_store/; data enters from data/bids_input/; outputs land in outputs/. **Data flow:** config + BIDS paths -> command generation -> subprocess execution -> logs -> diagnosis -> command repair/retry -> output QA -> final console report.

How to Run

- Create and activate a Python virtual environment.
- Install the packages named in README.md: langchain-groq, langgraph, sentence-transformers, nibabel, docker, and chromadb. Exact dependency manifest: Not found in repo.
- Place a valid license.txt in the repo root and ensure Docker is running.
- Keep the local manuals in data/docs/ and BIDS input in data/bids_input/ (both are present in this repo snapshot).
- Set GROQ_API_KEY if using the default Groq-backed configuration, then run python main.py.

Repo gaps explicitly marked: tests/CI setup not found in repo; a packaged dependency file such as requirements.txt or pyproject.toml not found in repo.